

Bijak Trust & Credit Engine: Technical System Architecture

Improvement Recommendations & Gap Analysis

Anjan Mahapatra

May 10, 2026

Abstract

This document provides a comprehensive technical architecture review of the Bijak Trust & Credit Engine platform, identifying gaps and recommending improvements across seven key dimensions: trust score computation resilience, credit underwriting latency, offline-capable retailer experience, MLOps for fraud detection, cost optimization, security enhancement, and observability. The platform currently serves 150,000+ retailers with 10,000+ daily transactions but faces scalability challenges. Based on analysis of 18 distinct gaps, we propose targeted improvements with prioritization (P0-P2). The target outcomes include: improving system availability from 99.9% to 99.99%, reducing credit decision latency from 15 seconds to under 2 seconds (p95), enabling offline functionality for rural retailer connectivity scenarios (where network uptime is only 40-60%), and achieving sub-second response times for 95% of Bijak Score queries. Total estimated effort is 56 person-weeks with infrastructure cost savings of 35% and ROI within 6-9 months.

Keywords: B2B marketplace, trust score, credit underwriting, offline-first architecture, MLOps, federated learning, fraud detection, distributed tracing, active-active deployment, rural connectivity, Bijak.

Contents

1	Executive Summary	3
1.1	Investment Summary	3
2	Architecture Overview: Current State Assessment	3
2.1	Detailed Gap Analysis	4
3	Critical Improvements (P0 - Immediate, 0-3 Months)	5
3.1	Offline-First Architecture for Retailers	5
3.1.1	Edge Score Cache Architecture	5
3.2	Enhanced Circuit Breaker with Fallbacks	5
3.3	MLOps: Fraud Detection Model Versioning & Drift Detection	6
3.3.1	Drift Detection with PSI	6
4	High-Priority Improvements (P1 - 3-6 Months)	7
4.1	Cost Optimization Strategy	7
4.1.1	Model Cascade Architecture for Fraud Detection	7
4.2	Enhanced Observability: Distributed Tracing	7
4.3	Multi-Region Active-Active Deployment	7
5	Medium-Priority Improvements (P2 - 6-12 Months)	8
5.1	Feature Store for Credit Scoring	8
5.2	Federated Learning for Privacy-Preserving Fraud Detection	8
5.3	WebSocket for Real-time Trust Updates	8
6	Security Enhancements	9
6.1	mTLS for Internal Services	9
6.2	Comprehensive Audit Trail	9
7	Capacity Planning for 1M+ Retailers	9
8	Risk Mitigation: Architecture-Specific	9
9	Implementation Roadmap	9
10	Conclusion	9
10.1	Critical Immediate Actions (Next 30 Days)	9
10.2	Expected Outcomes	10
10.3	Final Remarks	10

1 Executive Summary

The Bijak Trust & Credit Engine platform is well-designed for current scale (150K retailers, 10K daily transactions) but requires significant enhancements for target scale (1M+ retailers, 100K+ daily transactions) and rural Indian connectivity conditions. This document identifies 18 specific improvement areas across the platform architecture.

Table 1: Key Findings Summary by Priority

Priority	# of Gaps	Effort (weeks)	Primary Impact	ROI
P0 (Critical)	7	22	Retailer trust & fraud prevention	High
P1 (High)	6	16	Cost reduction & scalability	Medium-High
P2 (Medium)	5	18	Advanced capabilities	Medium
Total	18	56	Comprehensive	High

1.1 Investment Summary

Total estimated engineering effort is 56 person-weeks, costing approximately \$280,000 to \$420,000. Infrastructure costs are expected to decrease by 35% through optimization strategies including spot instances and model cascading. System availability is targeted to improve from 99.9% to 99.99%, credit decision latency from 15 seconds to under 2 seconds (p95), and Bijak Score query latency from 800ms to 200ms (p95).

Table 2: Investment Summary

Metric	Value
Total Estimated Effort	56 person-weeks
Total Estimated Cost (Engineering)	\$280,000 - \$420,000
Infrastructure Cost Impact	-35% (savings)
Expected Availability Improvement	99.9% → 99.99%
Expected Credit Decision Latency	15s → 2s (p95)
ROI Timeline	6-9 months

2 Architecture Overview: Current State Assessment

The Bijak platform follows a four-layer architecture. The Channel Layer (Mobile App & Web) currently has medium maturity with gaps in offline score access and multi-channel synchronization. The Gateway Layer (API Gateway & Load Balancers) shows medium-high maturity but lacks circuit breaker fallbacks and mTLS. The Service Layer (microservices including Credit Scoring, Fraud Detection, Bijak Score, Payment, KYC) has medium maturity with significant gaps in MLOps for fraud detection and cost optimization. The Data Layer (PostgreSQL, Redis, Kafka, S3) has medium-high maturity with gaps in audit trail and federated learning.

Table 3: Layer-wise Maturity Assessment

Layer	Maturity	P0 Gaps	P1 Gaps	P2 Gaps
Channel (Mobile/Web)	Medium	Offline score access	Multi-channel sync	Progressive Web App
Gateway	Medium-High	Circuit breaker fallbacks	Rate limiting	mTLS
Service	Medium	MLOps fraud detection	Cost optimization	Feature store
Data	Medium-High	—	Audit trail	Federated learning

2.1 Detailed Gap Analysis

The most critical gaps are in offline support. Currently, Bijak Score access requires online connectivity, which fails in rural areas where network uptime is only 40-60%. The target state is local encrypted cache with background synchronization. Transaction recording is currently server-only; the target is SQLite with CRDT-based sync for zero data loss. In resilience, circuit breakers currently use basic 3-5 failure thresholds without exponential backoff. The target is multi-level fallbacks with configurable timeouts. In MLOps, fraud detection models have no versioning and no drift detection. The target is MLflow registry with canary deployments and hourly PSI-based drift monitoring.

Table 4: Gap Analysis Matrix

Gap Category Priority	Specific Gap	Current State	Target State
Offline Support P0	Bijak Score access	Online-only	Local cache + sync
Offline Support P0	Transaction recording	Server-only	SQLite + CRDT sync
Resilience P0	Circuit breaker fallbacks	Basic (3-5 failures)	Multi-level fallbacks
MLOps P0	Fraud detection versioning	Manual	MLflow registry
MLOps P0	Model drift detection	None	PSI monitoring
Cost Optimization P0	GPU utilization	Always-on	Spot + on-demand
Observability P1	Distributed tracing	None	Jaeger + 1% sampling
Scalability P1	Multi-region	Active-passive	Active-active
Security P1	mTLS	None	Istio service mesh

3 Critical Improvements (P0 - Immediate, 0-3 Months)

3.1 Offline-First Architecture for Retailers

Problem Statement: The architecture assumes always-on connectivity for Bijak Score queries and transaction recording. Rural retailers in Bihar, UP, and Madhya Pradesh experience 40-60% network uptime, with 2G/3G speeds of 5-10 Mbps. According to TRAI data, packet loss during peak hours reaches 30-40%.

Impact Analysis: Approximately 35% of retailers experience session drops due to connectivity issues. An estimated 15-20% of potential orders are abandoned. Retailers cannot verify counterparty scores offline, leading to cash-only transactions that defeat the platform’s purpose.

Table 5: Offline-First Architecture Components

Component	Current	Proposed	Effort	Success Metric
Bijak Score Access	Online-only	Local encrypted cache + sync	2 weeks	90% score access offline
Transaction Recording	Real-time	Queue-based + CRDT sync	3 weeks	Zero data loss
Authentication	Always online	Offline-capable JWT + cache	1 week	30s offline auth
Order History	Server-only	Local SQLite + sync	2 weeks	Full history offline
Credit Application	Online-only	Offline form + batch submit	2 weeks	100% application capture

3.1.1 Edge Score Cache Architecture

The Bijak Score (0-1000) for trusted counterparties must be available offline using a time-decay model: $\text{Score_cache} = \text{Score_last_synced} \times (0.95)^{\text{age_days}}$. For example, a score of 800 that is 3 days old would be cached as $800 \times 0.857 = 686$.

Table 6: Edge Cache Configuration

Parameter	Value	Rationale
Cache TTL (trusted parties)	7 days	Frequent trading partners (10+ transactions)
Cache TTL (new parties)	24 hours	Higher risk; requires fresh verification
Max cached retailers	500	Storage constraint (50MB on device)
Sync frequency (WiFi)	Every 6 hours	Overnight sync when retailer at home
Sync frequency (Cellular)	Every 24 hours	Save mobile data costs
Encryption	AES-256 at rest	Prevents score tampering

3.2 Enhanced Circuit Breaker with Fallbacks

Problem Statement: Current circuit breakers (3-5 failures) lack exponential backoff and fallback strategies. A single credit scoring service outage currently impacts 100% of loan applications. During peak hours (10 AM - 2 PM), credit scoring failures affect up to 500 loan applications per hour.

Table 7: Circuit Breaker Configuration by Service

Service	Threshold	Timeout	Fallback	SLA
Credit Scoring	5/30s	5s	Rule-based score + 10% discount	99.99%
Fraud Detection	3/10s	3s	Manual review queue (4h)	99.9%
Bijak Score API	5/30s	2s	Cached score ($\leq 7d$ old)	99.99%
Payment Gateway	3/10s	15s	Queue + SMS retry	99.95%
KYC Verification	5/60s	10s	Deferred verification (24h)	99.9%

3.3 MLOps: Fraud Detection Model Versioning & Drift Detection

Problem Statement: No model versioning, canary deployments, or automated retraining for fraud detection models. Fraud pattern drift is undetected for weeks. In Q3 2025, a new fraud pattern (circular trading among 50 retailers) went undetected for 19 days, causing estimated loss of 25 lakh.

Table 8: MLOps Pipeline Components

Component	Technology	Purpose	Frequency	SLA
Feature Store	Feast + Redis	Feature versioning, reuse	Real-time	10ms
Model Registry	MLflow	Version tracking, staging	Per release	<5 min
Canary Deploy	Istio + Flagger	5% $\rightarrow$ 25% $\rightarrow$ 100% rollout	Per release	<1 min roll-back
Model Monitor	Evidently AI	Drift detection (PSI)	Hourly	<1 min
Auto-retraining	Kubeflow	Trigger on drift >0.2	Daily/Weekly	<4 hrs

3.3.1 Drift Detection with PSI

Population Stability Index measures distribution shift using the formula: $PSI = \sum_i \left(\frac{Actual_i - Expected_i}{Expected_i} \times \ln \left(\frac{Actual_i}{Expected_i} \right) \right)$. Drift thresholds are as follows. PSI below 0.1 requires no action. PSI between 0.1 and 0.15 schedules retraining in 2 weeks with Slack notification. PSI between 0.15 and 0.20 requires retraining within 1 week. PSI between 0.20 and 0.25 requires immediate retraining. PSI above 0.25 requires rollback to previous version with critical page.

Table 9: Drift Thresholds and Actions

PSI Range	Severity	Action	Notification
<0.1	None	Continue monitoring	None
0.1 - 0.15	Low	Schedule retraining in 2 weeks	Slack #ml-alerts
0.15 - 0.20	Medium	Retrain within 1 week	PagerDuty (non-urgent)
0.20 - 0.25	High	Retrain immediately	PagerDuty (urgent)
>0.25	Critical	Rollback to previous version	Critical page + SMS

4 High-Priority Improvements (P1 - 3-6 Months)

4.1 Cost Optimization Strategy

Problem Statement: Always-on GPU instances for fraud detection cost approximately \$8,000/month at current scale. Sixty-five percent of capacity is idle during non-peak hours (10 PM to 6 AM).

Table 10: GPU Workload Placement Strategy

Workload	Instance Type	Interruptible	Savings
Fraud Detection (real-time)	On-demand (g4dn.xlarge)	No	0%
Fraud Detection (batch)	Spot (g4dn.xlarge)	Yes	65%
Model Training	Spot (g4dn.2xlarge)	Yes (checkpoint)	70%
Nightly Feature Aggregation	Spot + Fallback	Yes	60%
Credit Scoring Batch	Spot (c5.2xlarge)	Yes	65%

4.1.1 Model Cascade Architecture for Fraud Detection

Instead of running the full ensemble model (5 million parameters, 50ms inference) for every transaction, a three-level cascade is implemented. Level 1 (Logistic Regression, 50K parameters) processes 100% of traffic. Level 2 (XGBoost, 500K parameters) processes approximately 35% of traffic. Level 3 (Graph Neural Network, 5M parameters) processes approximately 10% of traffic.

Table 11: Model Cascade Architecture

Level Traffic	Model	Params	Confidence Threshold
L1 (Small) 100%	Logistic Regression	50K	Confidence <80% → L2
L2 (Medium) 35%	XGBoost	500K	Confidence <90% → L3
L3 (Large) 10%	Graph Neural Network	5M	Always final

Cost savings calculation: Without cascade, cost is \$1.00 per 100 requests. With cascade, cost is \$0.305 per 100 requests, representing 69.5% savings. At current scale of 10,000 requests per day, monthly savings are approximately \$2,085.

4.2 Enhanced Observability: Distributed Tracing

Problem Statement: Prometheus and Grafana provide metrics but no end-to-end request tracing. Debugging slow credit decisions takes hours of log correlation.

4.3 Multi-Region Active-Active Deployment

Current Gap: Active-passive deployment with Mumbai primary and Hyderabad disaster recovery. Recovery Time Objective (RTO) is 30 minutes.

Table 12: Tracing Configuration

Parameter	Sampling Rate	Daily Storage	Rationale
Normal requests	1%	50 MB	Balance cost and visibility
Errors	100%	10 MB	Capture all failures
Slow requests (>2s)	100%	20 MB	Identify bottlenecks
Retention & Limits			
Trace retention	15 days (1.2 GB total) - debugging + compliance		
Max spans per trace	100 - prevent storage explosion		

Table 13: Active-Active Configuration

Aspect	Current (Active-Passive)	Proposed (Active-Active)
Traffic Distribution	100% Mumbai	70% Mumbai, 30% Hyderabad
Database	Primary-Replica (15 min lag)	Multi-master (Aurora Global)
RTO (Recovery Time Objective)	30 minutes	<1 minute (automatic)
RPO (Recovery Point Objective)	15 minutes	<5 seconds
Latency (East India)	80-100 ms	30-40 ms (routed to Hyderabad)
Failover Type	Manual	Automatic (Route 53)
Cost Premium	0%	25-30%

5 Medium-Priority Improvements (P2 - 6-12 Months)

5.1 Feature Store for Credit Scoring

Problem Statement: Credit scoring model uses 60+ features but no centralized feature store. Features are recomputed for every request, wasting compute. Current feature computation takes 800ms per request.

Table 14: Feature Store Components

Component	Technology	Purpose	Latency SLA
Online Feature Store	Redis + Feast	Low-latency inference	<10ms p99
Offline Feature Store	S3 + Parquet + Spark	Batch training, backfills	1 hour for 1M retailers
Feature Registry	Feast UI / YAML	Feature definitions, versioning	Real-time
Materialization Engine	Spark Streaming	Real-time feature updates	<1 minute

5.2 Federated Learning for Privacy-Preserving Fraud Detection

Problem Statement: All retailer transaction data is centralized for model training, creating privacy risk and regulatory compliance challenges with India's DPDP Act 2023.

5.3 WebSocket for Real-time Trust Updates

Problem Statement: REST APIs with polling for Bijak Score updates are inefficient and have high latency. Current polling uses 5-second intervals, generating 12,000 requests per active retailer per day.

Table 15: Privacy Guarantees

Guarantee	Value	Rationale
Differential Privacy	$\epsilon = 2.0, \delta = 1e - 5$	Strong privacy guarantee
Secure Aggregation	Homomorphic encryption (CKKS scheme)	Gradients unreadable individually
Data Locality	No raw data leaves device	Compliance with DPDP Act
Minimum Participants	100	Anonymization threshold
Dropout Resilience	30% client dropout tolerance	Rural network issues

Table 16: REST vs WebSocket Comparison

Metric	REST (Current)	WebSocket (Proposed)
Latency (p95)	800 ms	150 ms
Requests per retailer per day	12,000 (polling)	1 (connection)
Bidirectional	No	Yes
Real-time Score Updates	Polling-based (5s delay)	Push-based (<100ms)
Mobile Battery Impact	High (constant polling)	Low (single connection)

6 Security Enhancements

6.1 mTLS for Internal Services

Problem Statement: TLS 1.3 is used for external traffic only. Internal service-to-service communication uses plain HTTP, exposing the platform to man-in-the-middle attacks.

Table 17: mTLS Implementation Matrix

Service Pair	Method	Justification	Pri
API Gateway → Credit Service	mTLS+JWT	PII exposure (scores, financials)	P1
API Gateway → Fraud Detection	mTLS+JWT	High sensitivity, fraud flags	P1
Kafka → Producers/Consumers	SASL/SCRAM+mTLS	Production PII data stream	P1
Service → Service (internal)	Istio mTLS	Zero code change, all traffic	P1

6.2 Comprehensive Audit Trail

7 Capacity Planning for 1M+ Retailers

8 Risk Mitigation: Architecture-Specific

9 Implementation Roadmap

10 Conclusion

10.1 Critical Immediate Actions (Next 30 Days)

Implement offline-first Bijak Score access with local encrypted cache (3 weeks). Add circuit breaker fallbacks and exponential backoff for credit and fraud services (2 weeks). Deploy MLOps pipeline for fraud detection with versioning and drift monitoring (4 weeks). Optimize GPU costs with spot instances and model cascading (3 weeks).

Table 18: Audit Trail Specifications

Element	Configuration	Purpose
Correlation ID	UUID v4, W3C Trace-Context header	End-to-end request tracking
Log Retention (Hot)	30 days in Elasticsearch	Debugging
Log Retention (Cold)	7 years in S3 Glacier	Compliance (RBI, DPDP Act)
Audited Actions	All PII access, credit decisions, fraud alerts	Security
Tamper Protection	Write-once, append-only (CloudTrail Lake + immutable S3)	Forensics
Alert on Anomaly	5 failed logins, 10 PII accesses in 1 hour	Real-time detection

Table 19: Capacity Projections with Growth Phases

Component	150K Users	500K Users	1M Users	Scale Factor
API Pods	8	25	50	6x
Credit Scoring Pods	4	12	25	6x
Fraud Detection Pods	3	10	20	7x
RDS (vCPU)	4	16 (cluster)	32 (cluster)	8x
Redis Memory	10 GB	50 GB	150 GB	15x
Kafka Brokers	3	6	12	4x
Daily Transactions	10K	35K	100K	10x
Monthly Cost (USD)	8,000	35,000	80,000	10x

10.2 Expected Outcomes

10.3 Final Remarks

This architecture improvement plan provides a production-ready blueprint for scaling the Bijak Trust & Credit Engine platform. The total estimated effort of 56 person-weeks (\$280,000-\$420,000) is expected to deliver ROI within 6-9 months through infrastructure cost savings (35%), reduced fraud losses (0.4% of GMV), increased retailer retention (+10-15%), and faster credit decisions enabling higher loan volume. The platform's trust score and credit engine are foundational to Bijak's value proposition. These architectural improvements will ensure that Bijak remains the most trusted B2B agri-input marketplace in India as it scales from 150,000 to over 1,000,000 retailers.

Table 20: Architecture Risk Register with RPN

Risk	P	I	RPN	Mitigation
Bijak Score API rate limiting	3	4	12	Queue + exponential backoff, SMS fallback
Fraud pattern drift	3	4	12	Hourly PSI monitoring, auto-retraining
Vendor lock-in (AWS, Twilio, Kafka)	3	4	12	Abstraction layers, multi-provider readiness
Retailer PII data breach	2	5	10	AES-256 encryption, SOC 2 audit
Database connection pool exhaustion	3	3	9	PgBouncer, read replicas

Table 21: Four-Phase Implementation Roadmap

Phase	Timeline	Key Deliverables	Success Metrics
Phase 1: Foundation	Weeks 1-7	Offline score cache, Circuit breaker fallbacks, mTLS	90% offline access, P99 latency <2s
Phase 2: Intelligence	Weeks 8-14	MLflow registry, Drift detection, Auto-retraining	Drift detection <1hr, 30% cost reduction
Phase 3: Scale	Weeks 15-20	Active-active multi-region, Feature store, Tracing	RTO <1 min
Phase 4: Advanced	Weeks 21-26	Federated learning, Web-Socket, Audit trail	99.99% avail, 150ms p95 latency

Table 22: Expected Outcomes Summary

Metric	Current	Target (Post-Implementation)
Availability (uptime)	99.9%	99.99%
Offline Bijak Score access	0%	90% of queries served offline
Infrastructure cost	Baseline	-35%
Credit decision latency (p95)	15 seconds	2 seconds
Fraud detection drift detection	Manual (weeks)	Automated (<1 hour)
Model update time	2-4 weeks (manual)	1 day (automated)
Fraud loss rate	1.2% of GMV	0.8% of GMV